Feature Area	Component	User Story	Scenario	Acceptance Criteria	Trigger/Interaction	Expected Behavior	Status
Backend ML	SageMaker Endpoint	As a job seeker, I want accurate and fast job recommendations based on my resume, so that I can quickly discover relevant job opportunities.	Endpoint accepts resume input and returns prediction	The trained ML model is deployed as a SageMaker endpoint that accepts resume input and returns job recommendations.	Send a valid resume payload to the SageMaker HTTPS endpoint	HTTP 200 with JSON including predicted job category and a list of job recommendations	Pass
Backend ML	SageMaker Endpoint	...	Endpoint requires secure access	The endpoint is accessible via a secure HTTP API.	Invoke endpoint over HTTPS with and without valid auth/credentials	HTTPS succeeds with valid credentials; unauthorized requests receive 401/403	Pass
Backend ML	SageMaker Endpoint	...	Typical prediction completes under 1 second	The model returns predictions within 1 second for a typical resume file.	Post a typical-size resume (≤200KB PDF/text) and measure latency	p95 latency ≤ 1000 ms for response	Pass
Backend ML	SageMaker Endpoint	...	Output contains IDs, scores, and metadata	Model output includes a list of job IDs, match percentages, and any additional metadata used by the frontend.	Send a valid resume and inspect response schema	Response JSON includes non-empty list of objects with jobId, matchPct, and metadata fields	Pass
Backend ML	SageMaker Endpoint	...	Monitoring and graceful error handling in place	Model uptime is monitored, and errors are logged or handled gracefully.	Induce a handled error (e.g., malformed payload) and check logs/metrics	Request returns a clear error (4xx/5xx as appropriate); error is logged; endpoint health remains OK	Pass
Backend API	Flask Endpoint	As a job seeker, I want the app to process my uploaded resume and return job matches, so that I can get personalized recommendations without needing technical knowledge.	/process-resume accepts file upload	Flask API exposes an endpoint /process-resume that accepts file uploads via POST.	POST multipart/form-data with 'file' to /process-resume	HTTP 200; response body includes resumeID and list of matched jobs	Pass
Backend API	Flask Endpoint	...	Resume forwarded to ML model and results returned	The uploaded resume is forwarded to the ML model and the results are returned in JSON format.	Mock/stub model to return known IDs; call /process-resume	Response JSON contains the same job IDs and match data from the model	Pass
Backend API	Flask Endpoint	...	Response contains resumeID and matched jobs	API response includes a resumeID and list of matched jobs.	Call /process-resume successfully	Response includes non-empty resumeID string and matches array	Pass
Backend API	Flask Endpoint	...	CORS enabled for frontend domain	CORS is enabled so the frontend can make cross-origin requests.	Browser preflight and POST from frontend origin	CORS headers present; preflight and POST succeed without CORS errors	Pass
Backend API	Flask Endpoint	...	Correct status codes on success and errors	Returns appropriate status codes for success (200), client errors (400), or server errors (500).	Submit valid request; missing file; trigger server error	Valid returns 200; missing/invalid returns 400; server fault returns 500 with safe error body	Pass
Frontend	Browser Compatibility	As a user, I should be able to visit and use the website from any browser without errors.	Load and basic navigation across browsers	Ensure ResuMind works across browsers.	Open app in latest Chrome, Safari and navigate Home → Search → Matches	Each page loads and is usable with no blocking console errors	Pass
Frontend	Browser Compatibility	...	File upload works in common browsers	Ensure file upload works across browsers.	Upload PDF/DOC/DOCX in each browser	Valid files are accepted and processed consistently; invalid types show validation message	Pass
Frontend	Browser Compatibility	...	Responsive layout remains intact	Website looks correct across screen sizes.	Resize viewport to 1440, 1024, 390 widths	No overlapping/clipped text; navbar/cards remain accessible	Pass
Frontend	Browser Compatibility	...	Swipe and click interactions behave consistently	Swiping/clicking works in all browsers.	Swipe cards and click actions on each browser	Interactions execute with the same behavior and visual feedback	Pass
Frontend	Form Inputs	As a new user, I want to enter my name and upload my resume easily, so that I can quickly get started and receive job suggestions personalized to me.	Homepage shows name field and resume upload	The homepage includes a text input for name and a resume upload button.	Load homepage	Name input and upload control are visible and focusable	Pass
Frontend	Form Inputs	...	Accept only supported file types	The resume input accepts .pdf, .doc, and .docx files.	Try uploading .pdf/.doc/.docx then .txt/.png	Supported types accepted; unsupported types blocked with a friendly message	Pass
Frontend	Form Inputs	...	Selected file stored and ready to send	Upon selection, the file is stored and ready to be sent to the backend.	Select a valid file and inspect client state	File object is present in state and attached for submission	Pass
Frontend	Form Inputs	...	Optional name personalizes UI text	The name input can be optionally used to personalize UI text.	Enter a name and continue; repeat with empty name	Greeting/personalized text appears when provided; app still works without a name	Pass
Frontend	Backend Call	As a job seeker, I want my resume to be analyzed as soon as I upload it, so that I can begin swiping through job matches without delay.	Auto POST to /process-resume on file select	The frontend calls the Flask backend /process-resume endpoint when a resume file is selected.	Choose a resume file on the upload control	Frontend sends POST /process-resume and shows loading state	Pass
Frontend	Backend Call	...	Save resumeID for session tracking	On success, a resumeID is saved to local storage for session tracking.	Complete a successful upload and inspect storage	resumeID is saved in localStorage/sessionStorage	Pass
Frontend	Backend Call	...	Navigate to search after success	If successful, the user can navigate to the /search page to begin swiping.	After receiving a 200 response	App routes to /search and job cards are visible	Pass
Frontend	Job Card Component	As a job seeker, I want each job card to clearly display relevant job details (title, description, match score, etc.), so that I can make informed decisions while swiping.	Render with all expected props	The job card component accepts props for title, description, matchScore, and likelihood.	Render a card with those four props	Each field displays correctly; no overflow/clipping	Pass
Frontend	Job Card Component	...	Props populated from backend data	Props are dynamically populated from job data received from the backend.	Load a jobs response and map to card props	Cards show correct values corresponding to data	Pass
Frontend	Job Card Component	...	Graceful fallbacks when data missing	Cards render correctly with various prop values and show placeholder text if a prop is missing.	Omit one prop at a time and render	Placeholder text appears; card does not crash	Pass
Frontend	Job Card Component	...	Reusable across pages	The component is reusable across the swipe and matches pages.	Render the same component on Swipe and Matches views	Visuals/behavior consistent across both pages	Pass
Frontend	Job Card Component	...	Test card renders during development	A test card renders correctly with mock data during development.	Render with mock job data	Card renders as designed with mock content	Pass
Backend	Job Dataset Query	As a job seeker, I want the platform to match me with real job postings based on my resume, so that the recommendations I receive are relevant and up-to-date.	Backend connects to configured dataset	Backend connects to a structured dataset (CSV, database, or API) of job postings.	Startup/connect and perform a sample query	Query returns ≥1 job from configured source	Pass
Backend	Job Dataset Query	...	Retrieve relevant jobs by skills/keywords/embedding	A function is implemented to retrieve relevant jobs based on skills, keywords, or embedding similarity.	Request recommendations with known skills in resume	Top results include those skills or close semantic matches	Pass
Backend	Job Dataset Query	...	Only verified or curated jobs are returned	Only verified or curated jobs are returned.	Include an unverified item in test data and query	Unverified item is not present in results	Pass
Backend	Job Dataset Query	...	Return at least 20 jobs per request	At least 20 jobs are returned per resume request.	Perform a standard resume request	Response includes 20 or more job objects (or first page size ≥20)	Pass
Backend	Job Dataset Query	...	Each returned job has required fields	Returned job objects include id, title, description, and metadata used by the frontend.	Inspect one returned object	Object includes id, title, description, and metadata fields	Pass
Project Setup	Test Case Sheet	As a developer, I want a centralized test case sheet that outlines how each feature will be tested, so that I can systematically verify functionality and ensure the app works as expected.	Current stories have tests	Test cases are written for the currently implemented user stories.	Review the sheet rows	Each sprint story has at least one corresponding test case	Pass
Project Setup	Test Case Sheet	...	Sheet stored in shared location	The sheet is stored in a shared location (shared drive).	Open sheet from shared drive path	Path is shared; teammates can access it	Pass
Project Setup	Test Case Sheet	...	Team can contribute tests	The team has access and can contribute additional test cases as features are developed.	Teammate attempts to add/edit rows	Changes save successfully and appear in version history	Pass
Backend API	Process Resume	As a job seeker, I want the app to process my uploaded resume and return job matches, so that I can start browsing job options personalized to my background.	Accept multipart file uploads	A POST endpoint /process-resume accepts file uploads via multipart/form-data.	POST multipart/form-data with 'file' field	HTTP 200 with JSON {resumeID, matches:[...] }	Pass
Backend API	Process Resume	...	Extract resume content and pass to model	Resume content is extracted and passed to the ML model.	Upload a PDF containing a known keyword/skill	Extraction contains the keyword; payload sent onward to model	Pass
Backend API	Process Resume	...	Return resumeID and matched jobs	The endpoint returns a JSON response containing a resumeID and a list of matched jobs.	Successful request to /process-resume	Response includes non-empty resumeID and matches array	Pass

Feature Area	Component	User Story	Scenario	Acceptance Criteria	Trigger/Interaction	Expected Behavior	Status
Backend API	Process Resume	...	Validate format and size	Validation is included for file format and size.	Upload unsupported or oversized file	HTTP 400/413 with user-friendly validation message	Pass
Backend API	Process Resume	...	Graceful error responses	Errors return appropriate status codes with user-friendly messages.	Force extraction failure or model error	HTTP 500 with safe message; no stack traces leaked	Pass
Backend API	Model Inference	As a job seeker, I want the platform to generate job recommendations based on my resume's content, so that I can see how well I match open roles.	Accept resumeID and return predictions	A GET endpoint /model-inference accepts a resumeID as part of a body.	Call /model-inference with a resumeID (query/body to match implementation)	HTTP 200 with list of job IDs and their match scores	Pass
Backend API	Model Inference	...	Resume is fetched	The resume is fetched and passed to the ML model for inference.	Call the /model-inference endpoint with a valid resumeID that was previously generated by /process-resume	Model runs and returns job IDs with match scores	Pass
Backend API	Model Inference	...	Response includes job IDs and their match scores	The response contains a list of job IDs and their respective match scores.	Call /model-inference with a valid resumeID	JSON returns list of job IDs with match scores	Pass
Backend API	Model Inference	...	Run model inference	Model inference is completed and returned.	Invoke inference for a valid stored resumeID	Model runs successfully and returns scores	Pass
Project Setup	Deployment Research	As a developer working on ResuMind, I want to identify the best deployment strategy for the backend and ML model, so that users experience fast and reliable service.	Compare at least two methods	At least 2 deployment methods are evaluated (e.g., AWS SageMaker, Lambda, EC2)	Open the research report	Report shows evaluation of 2+ methods including cost and Flask integration	Pass
Project Setup	Deployment Research	...	Clear recommendation recorded	Recommendations are documented clearly in a markdown or PDF report.	Go to the conclusion of the report	One method is recommended with rationale and risks	Pass
Project Setup	Deployment Research	...	Team agreement for MVP	Team agrees on a method based on findings for MVP launch.	Check team notes/meeting outcomes	Decision captured and linked for reference	Pass